

Intro to Computational Math, Spring 2026
Section 11, April 21 (not due or graded).

1 Runge–Kutta methods

This will be a shorter section as the material is both rather straightforward and dense, making it not really very straight forward. By the former, I mean that the methods are entirely algebraic tricks that, like creating finite difference methods, rely on Taylor expansion and cleverly chosen parameters to achieve the highest order of accuracy. Unlike the finite difference methods, there isn't as simple of a trick as polynomial interpolation, and constructing these methods require some heavy mechanical work. Regardless, the idea is similar to tasks we have already done, and the result is similar as well: more nodes, more degrees of freedom, lead to higher convergence at the cost are increasingly complex (from the human side) methods.

The following methods are all still technically one step methods. We recall the definition below.

Definition 1.1

A **one-step method** for an IVP is a formula of the form

$$u_{i+1} = u_i + h\phi(t_i, u_i, h), \quad i = 0, \dots, n-1$$

We have seen an example of this before, where Euler's method uses $\phi = f(t_i, u_i)$. While many other methods can be written in a one-step form, they often get very complicated and cluttered, so for ease of implementation, we often write them in what looks like many steps. Really, the idea is setting up many helper functions, or intermediaries, so that our single step is some weighted combination of them. This is analog to finite difference methods in the sense of scaling and adding nodes.

Recall that for Euler's method, we set the next step u_{i+1} , an approximation of the true trajectory, by taking the first-order Taylor approximation at our current point (t_i, u_i) with the information that the true derivative follows some model $u'(t) = f(t, u(t))$. That is

$$\hat{u}(t_{i+1}) \approx u_{i+1} = u_i + \underbrace{f(t_i, u_i)(t_{i+1} - t_i)}_{\text{Euler Step}} = u_i + hf(t_i, u_i)$$

whenever our nodes are equally spaced by h . If instead we look at a second order Taylor expansion then

$$\hat{u}(t_{i+1}) \approx \hat{u}_i + h\hat{u}'(t_i) + \frac{1}{2}h^2\hat{u}''(t_i)$$

up to an $O(h^3)$ term. With the approximations that $u_i \approx \hat{u}(t_i)$, we can then use the replacement that $u'(t_i) = f(t_i, u_i)$ to set

$$u_{i+1} = u_i + hf(t_i, u_i) + \frac{1}{2}h^2u''(t_i).$$

The last term is simply $u'' = f'$ by definition. However, we have

$$f' = \frac{\partial f(t, u(t))}{\partial t} = \frac{\partial f}{\partial t} + \frac{\partial f}{\partial u} \frac{\partial u}{\partial t}$$

by the chain rule. But u is a single variable function of t , and $u' = f$ by again, our IVP, so this all collapses to

$$u''(t) = \frac{\partial f}{\partial t} + f(t, u) \frac{\partial f}{\partial u}.$$

Remark 1

It is worth the few minutes to go back and make sure you understand everything we did above. It is a decent amount of calculus and notational movement, but we did nothing more than Taylor expansion and applying some chain rule and substitutions. Notably we have

- $\hat{u}(t_{i+1}) \approx u_i + hu'(t_i) + \frac{1}{2}h^2u''(t_i)$
- $u'(t_i) = f(t_i, u_i)$
- $u''(t_i) = \frac{\partial f}{\partial t} + f(t_i, u_i)\frac{\partial f}{\partial u}$

All this together lets us define a decent approximation (with some notational abuse)

$$u_{i+1} = u_i + h \left[f(t_i, u_i) + \frac{h}{2} \frac{\partial f(t_i, u_i)}{\partial t} + \frac{h}{2} f(t_i, u_i) \frac{\partial f(t_i, u_i)}{\partial u} \right].$$

We pull out the h to get this in the form of a one-step method.

Notably, that term in brackets also looks like a Taylor expansion (this time around the function f with respect to both variable. We are not doing the same thing as the chain rule on $u''(t)$ above). We simply need to find what to center, and where to move from that center, that leads to a term that has the bracketed value as a decent approximation. That is, we wish to find α and β such that

$$f(t_i + \alpha, u_i + \beta) \approx f(t_i, u_i) + \alpha \frac{\partial f(t_i, u_i)}{\partial t} + \beta \frac{\partial f(t_i, u_i)}{\partial u}.$$

If we pattern match with the bracketed term, we need to choose

$$\alpha = \frac{h}{2}, \quad \beta = \frac{h}{2} f(t_i, u_i).$$

Making this final substitution, we get

$$u_{i+1} = u_i + hf \left(t_i + \frac{h}{2}, u_i + \frac{h}{2} f(t_i, u_i) \right).$$

This is notably a bit of a weird method. We have this nested iteration of our function f . Regardless, it is technically step a one-step method, but we can implement it easily as one that saves a few intermediaries. This is called a *multistage method*.

Algorithm 1.1 (Improved Euler Method)

Given initial value problem $u' = f(t, u)$ for $a \leq t \leq b$, initial value u_0 , and stepsize $h > 0$

For $i = 0, 1, 2, \dots$

$$\begin{aligned} k_1 &= hf(t_i, u_i) \\ k_2 &= hf(t_i + \frac{1}{2}h, u_i + \frac{1}{2}k_1) \\ u_{i+1} &= u_i + k_2 \end{aligned}$$

Note that another way we can write the last line, to incorporate a true “summing and weighting the previous steps” is to say

$$u_{i+1} = u_i + \underbrace{0k_1 + \frac{1}{2}k_2}_{\text{forward step}}.$$

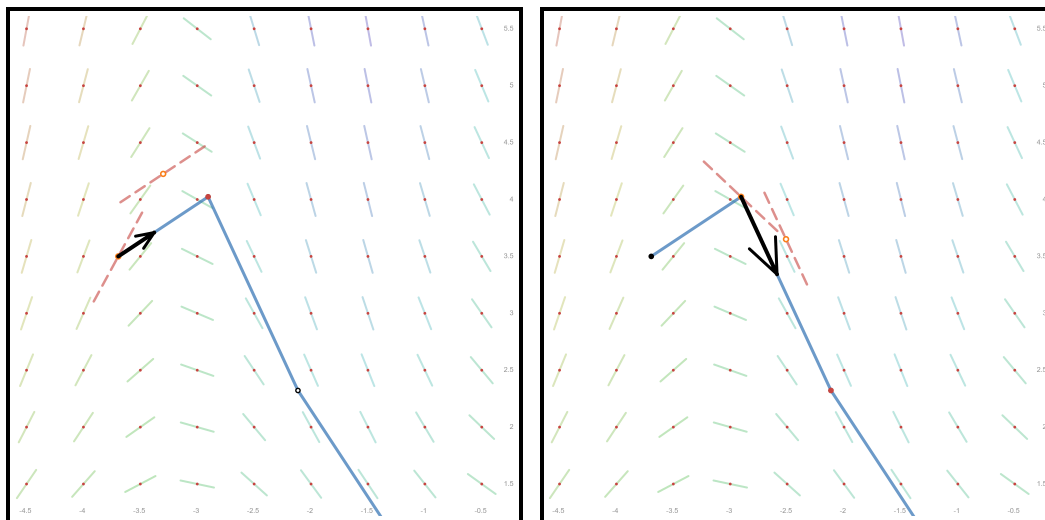
In fact, we can abstract away all the specific constants and write this out as

$$\begin{aligned} k_1 &= hf(t_i, u_i) \\ k_2 &= hf(t_i + c_1h, u_i + a_{11}k_1) \\ u_{i+1} &= u_i + \underbrace{b_1k_1 + b_2k_2}_{\text{forward step}} \end{aligned}$$

where our first stage analyzes the slope at our current point, the second stage analyzes the slope as some future spot (not necessarily on the trajectory) and then the final step does some type of weighting of the two.

Remark 2

The final iteration in these methods is the “forward step,” where we move from our current point onward. This forward step is a weighted sum of the previous stages, but it is not necessarily an average. We do in general want $b_1 + b_2 = 1$ or for higher order methods, $\sum_i b_i = 1$, so there is some notion of consistency, but notably we do not need $b_i \geq 0$. In fact, we can have methods that subtract some amount of k_i , while still being efficient and convergent.



Two steps of Improved Euler's Method

Recall that for Euler's method, we simply march along in the direction of our vector field. Improved Euler, and any two-phase method, behaves slightly different. Instead, we track first our velocity at our current point, but instead of moving in that direction, we set up a hypothetical movement. We save an intermediary position that moves in that direction and mark the velocity at that new point. These two velocities are then saved as k_1 and k_2 respectively. Finally, we move in a direction that is some combination of these velocities. Improved Euler puts all the weight on the second, intermediary direction, but other methods create more nontrivial combinations.

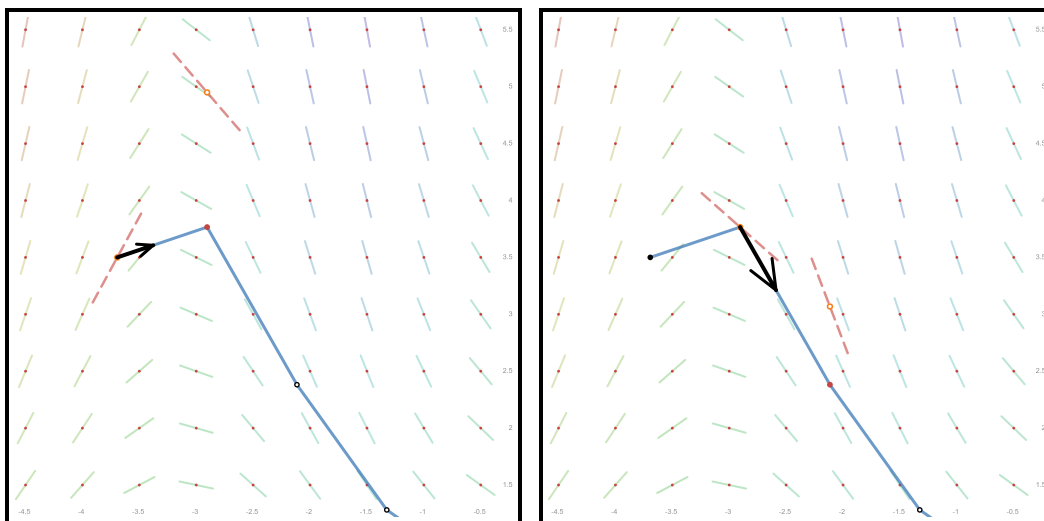
Algorithm 1.2 (Modified Euler Method)

Given initial value problem $u' = f(t, u)$ for $a \leq t \leq b$, initial value u_0 , and stepsize $h > 0$

For $i = 0, 1, 2, \dots$

$$\begin{aligned} k_1 &= hf(t_i, u_i) \\ k_2 &= hf(t_i + h, u_i + k_1) \\ u_{i+1} &= u_i + \frac{1}{2}k_1 + \frac{1}{2}k_2 \end{aligned}$$

This second method marches to an intermediary that is a full time step away, measures a velocity there and then averages the two for the true forward step.



Two steps of Modified Euler's Method

This third method does a more complex averaging. Note that more of the weight is on the forward intermediary, so this lies somewhere in between the two previous methods. That is, the forward step looks like it lies more in the intermediary's direction than the original. Regardless, all three methods have the same order of convergence, and all perform better than the basic Euler's method.

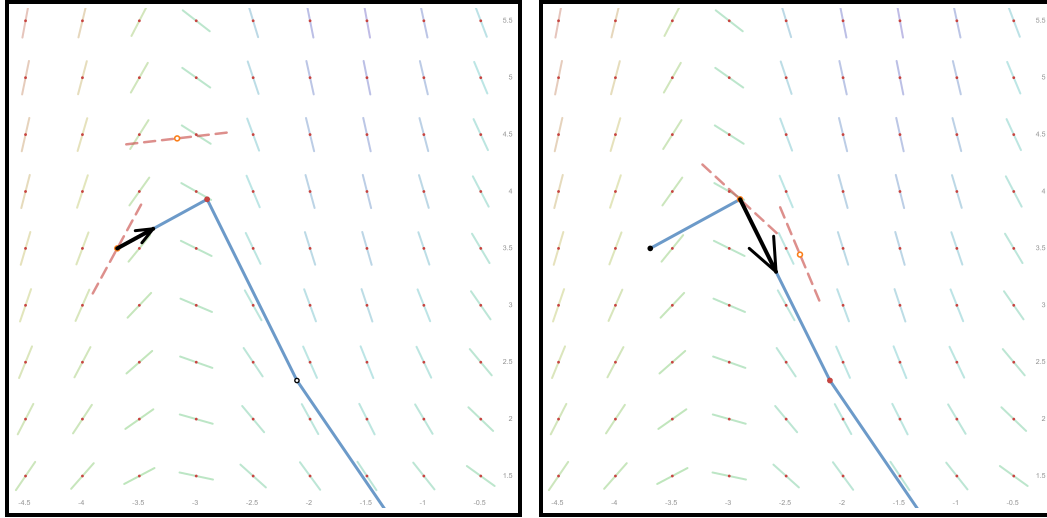
Algorithm 1.3 (Huen's/Ralston's Method)

Given initial value problem $u' = f(t, u)$ for $a \leq t \leq b$, initial value u_0 , and stepsize $h > 0$

For $i = 0, 1, 2, \dots$

$$\begin{aligned} k_1 &= hf(t_i, u_i) \\ k_2 &= hf(t_i + \frac{2}{3}h, u_i + \frac{2}{3}k_1) \\ u_{i+1} &= u_i + \frac{1}{4}k_1 + \frac{3}{4}k_2 \end{aligned}$$

The existence of these three methods that all look similar, but with different parameters leads



Two steps of Huen's/Ralston's Method

us to question if more exist. Further, since Huen's method looks as if it lies somewhere in between Improved and Modified Euler, we may seek some type of parameterization. Not too surprisingly, one exists, and its derivation once again comes from some Taylor expansion. One can play around with all these methods [in this interactive graph](#).

Algorithm 1.4 (General Two-Phase Method)

Given initial value problem $u' = f(t, u)$ for $a \leq t \leq b$, initial value u_0 , and stepsize $h > 0$

For $i = 0, 1, 2, \dots$

$$\begin{aligned} k_1 &= hf(t_i, u_i) \\ k_2 &= hf(t_i + \alpha, u_i + \alpha k_1) \\ u_{i+1} &= u_i + \left(1 - \frac{1}{2\alpha}\right)k_1 + \frac{1}{2\alpha}k_2 \end{aligned}$$

where $\alpha \in (0, 1]$

Notably, when $\alpha = \frac{1}{2}$, we recover Improved Euler, when $\alpha = 1$, we recover Modified Euler, and when $\alpha = \frac{2}{3}$, we recover Huen's/Ralston's Method.

Remark 3

Note: there seems to be some contradiction on the naming of these methods in various literature, so the more important aspect is understanding how they work and why.

1.1 General s -stage Runge Kutta Methods

A generic s -stage Runge Kutta method will take the following form.

Definition 1.2

Given real numbers $c_1, \dots, c_{s-1}$, $a_{i,j}$ for $1 \leq i \leq j \leq s-1$ and $b_1, \dots, b_s$, an s -stage method follows

$$\begin{aligned}
 k_1 &= hf(t_i, u_i) \\
 k_2 &= hf(t_i + c_1 h, u_i + a_{11} k_1) \\
 k_3 &= hf(t_i + c_2 h, u_i + a_{21} k_1 + a_{22} k_2) \\
 &\vdots \\
 k_s &= hf(t_i + c_{s-1} h, u_i + a_{s-1,1} k_1 + \dots + a_{s-1,s-1} k_{s-1}) \\
 u_{i+1} &= u_i + b_1 k_1 + \dots + b_s k_s
 \end{aligned}$$

We can interpret the method as ones where each phase analyzes a velocity at some future time with some value perturbed by the previous phases. Then, our forward step is some weighted combination of all the velocities witness. Since these methods are so constrained to the recipe of ingredients being the values of $a_{i,j}$, b_i , c_i we often write the method in a more compact form.

0				
c_1	a_{11}			
c_2	a_{21}	a_{22}		
c_3	a_{31}	a_{32}	a_{33}	
$\vdots$	$\vdots$		$\ddots$	
c_{s-1}	$a_{s-1,1}$	$a_{s-1,2}$	$\dots$	$a_{s-1,s-1}$
	b_1	b_2	$\dots$	b_s

This *Butcher's Tableau* completely defines the method and we can write out the algorithms above in these nice, compact forms.